

Lecture 16: More Virtual Memory

Professor G. Sandoval

16



Some slides adapted by G. Sandoval for CS3224,
from slide by Brendan Dolan-Gavitt

<Recap>

How to fix? Indirection!

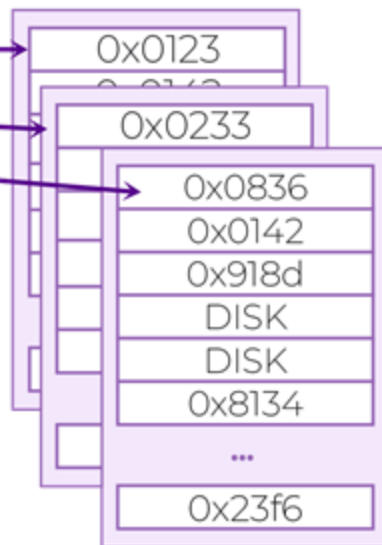
1st Level Page Table

4KB = 1024 PTEs



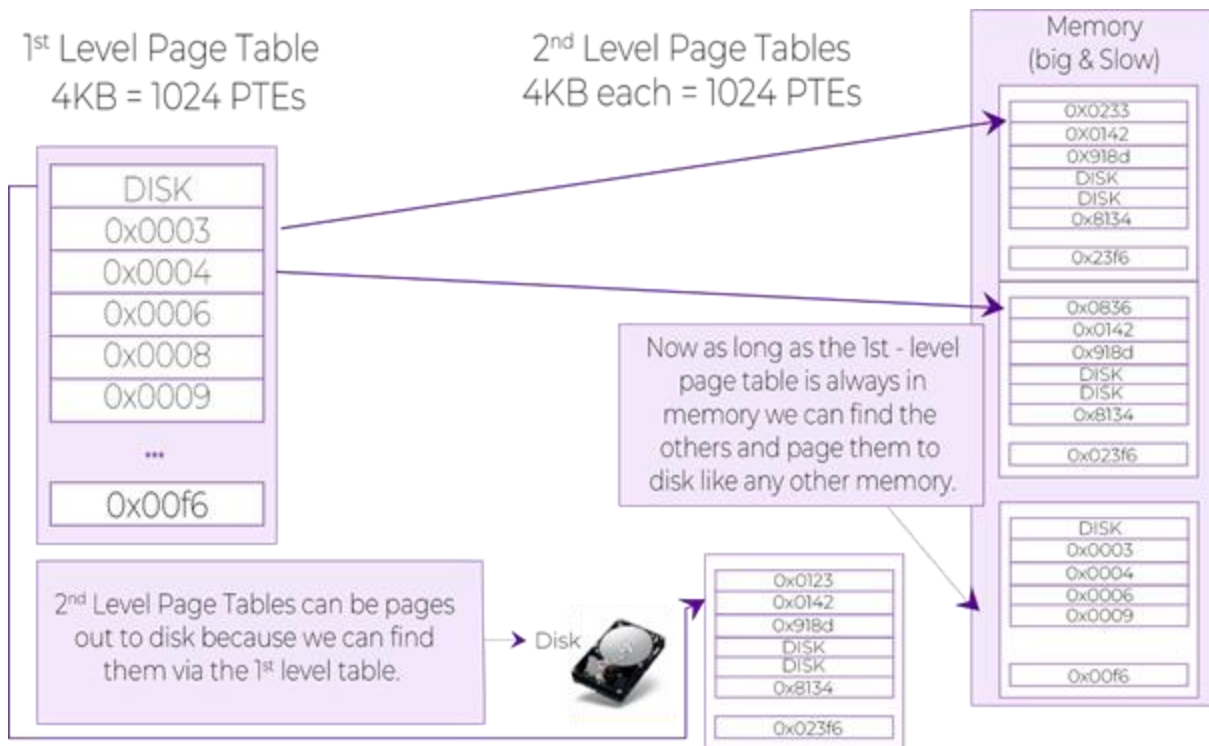
2nd Level Page Tables

4KB each = 1024 PTEs



Memory
(big & Slow)

Some paged into disk!



Page Table Size

- For 32-bit machine with 4kb pages we need:
 - 1M Page Table Entries
- (32 bits –
 - 12 bits for page offset =
 - 20 bits, $2^{20} = 1\text{M}$)
 - Each PTE is 4 bytes (20 bits for physical page + 12 permission bits)
 - 4 MB Total

Page Table Size

- Not bad...
 - Except each program needs its own page table
 - If we have 100 programs running, we need 400 MB of page tables.
- The worst part:
 - We can't swap the page tables out to disk
 - If the page table is not in RAM, we have no way to access it.

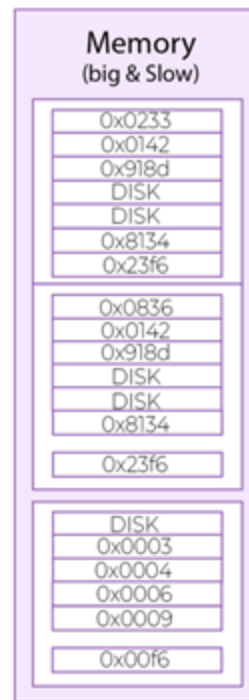
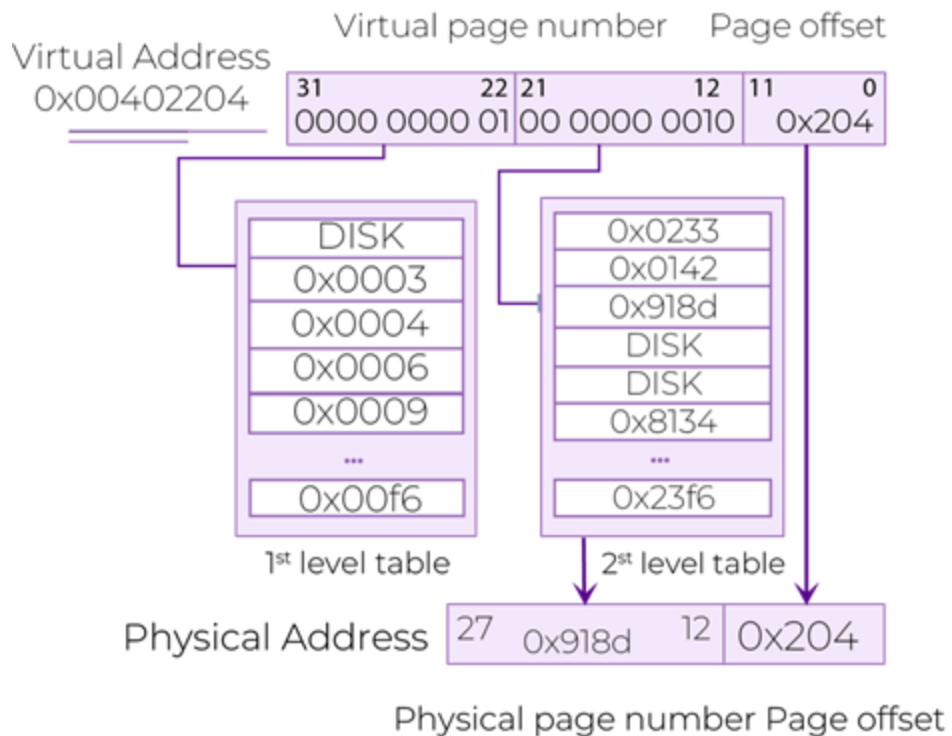
Example

- Q: With multi level page tables, what's the **smallest** amount of page table data we need to **keep in memory** for each 32 bit application to **access memory**?
 - A. 4 MB + 4 KB (always need the whole page table plus 1st level page)
 - B. 4MB (always need the whole page table in RAM)
 - C. 4 KB + 4KB (need the 1st level page plus one 2nd level page table)
 - D. 4 KB (just need the 1st level page table)

Class 16: Q: With multi level page tables, what's the smallest amount of page table data we need to keep in memory for each 32 bit application to access memory?

- 4 MB + 4 KB (always need the whole page table plus 1st level page) 0%
- 4MB (always need the whole page table in RAM) 0%
- 4 KB + 4KB (need the 1st level page plus one 2nd level page table) 0%
- 4 KB (just need the 1st level page table) 0%
- None of the above 0%

Address Translation



Example 2

- Q: If I'm running 100 applications on my computer with a 2 level page table, how much memory do I have to keep in RAM at all times to be able to access the application data?
 - A. 4 KB + 4 KB (need the 1st level page table and one 2nd level page table)
 - B. 400 KB (just need the first level page table for each application)
 - C. 800 KB (need the 1st and 2nd level page tables for each application)
 - D. 400 MB (need the full page table for each application)

Class 16: Q: If I'm running 100 applications on my computer with a 2 level page table, how much memory do I have to keep in RAM at all times to be able to access the application data?

4 KB + 4 KB (need the 1st level page table and one 2nd level page table)

0%

400 KB (just need the first level page table for each application)

0%

800 KB (need the 1st and 2nd level page tables for each application)

0%

400 MB (need the full page table for each application)

0%

None of the above

0%

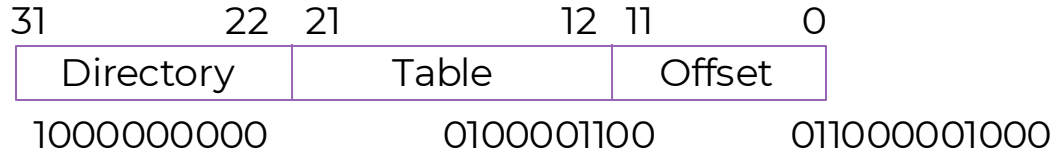
<Recap>

Worked Example

- Let's take an arbitrary address we got from running xv6: 0x8010c608
- In binary:
- CR3 = 0x003ff000

10000000000100001100011000001000

Worked Example



Directory index	=	1000000000	=	512
Table index	=	0100001100	=	268
Offset	=	011000001000	=	1544

Page Directory

Address Entry

CR3 = 0x003ff000

Directory index = 512

0x003ff000: 0x00000000
0x003ff004: 0x00000000
0x003ff008: 0x00000000
0x003ff00c: 0x00000000
0x003ff010: 0x00000000
[...]
0x003ff800: 0x003fe027
0x003ff804: 0x003fd027
0x003ff808: 0x003fc027
0x003ff80c: 0x003fb027
0x003ff810: 0x003fa027
0x003ff814: 0x003f9027
[...]

Note:

$0x003ff000 + 512 * 4$
 $= 0x003ff800$

Page Table Entry

0x003fe027

0000 0000 0011 1111 1110 0000 0010 0111

32	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Address of page table																				Ignored			Q	I G N	A	P C D	P W D	U R S	W /	W /	1	PED: Page Table

Address of page table: 0x3fe000

(A)

Accessed: Yes

(PCD)

Cache disabled: No

(PWT)

Write through caching: No

(U/S)

User-accessible: Yes

(R/W)

Read/Write: Yes

(P)

Present: Yes

Page Table

Address Entry

Address of page
table:
0x3fe0000



0x003fe000:	0x00000063
0x003fe004:	0x00001003
0x003fe008:	0x00002003
0x003fe00c:	0x00003003
0x003fe010:	0x00004003
[...]	
0x003fe430:	0x0010c063
0x003fe434:	0x0010d063
0x003fe438:	0x0010e063
0x003fe43c:	0x0010f063
0x003fe440:	0x00110063
0x003fe440:	0x00111063
[...]	

Table index = 268



Note:

$0x003fe000 + 268 * 4$
 $= 0x003fe430$

Page Table Entry

0x0010c063

000000000000100001100000001100011

32	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Address of page table																					Ignored			Q	I G N	A	P C D	P W T	U R S W	/	/	1	PED: Page Table		

Address of physical page: 0x10c000

(G)

Global: No

(PAT)

PAT page: No

(D)

Dirty: Yes

(A)

Accessed: Yes

(PCD)

Page Cache Disable: No

(PWT)

Page Write Through: No

(U/S)

User-accessible: No

(R/W)

Read/Write: Yes

(P)

Present: Yes

Physical Page

Address of page:
0x10c000

Offset = 1544 = 0x608

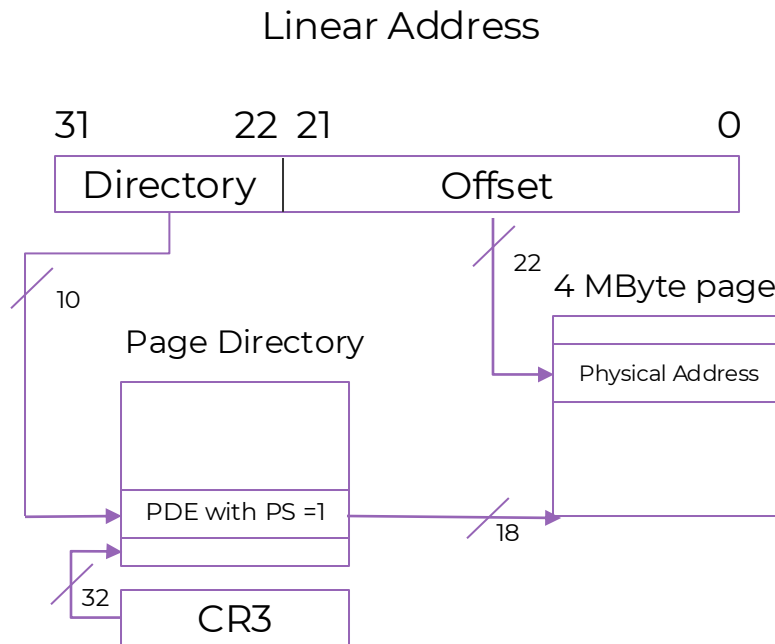
Offset = 1544 = 0x608

```
0000000: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000040: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000050: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000060: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000070: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000080: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000090: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000a0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
[...]
0000600: 8020 1180 8208 0000 28c6 1080 524a 1080 .).....(..RJ..
0000610: 3a87 ff07 0000 0000 38c6 1080 b448 1180 .....8....H..
0000620: 2824 1180 0100 0000 38c6 1080 8038 1080 ($.....8...8..
0000630: 0000 4080 0000 008e 48c6 1080 2138 1080 ..@.....H...!8..
0000640: 0000 0000 54c6 1080 f87b 0000 0000 0000 ....T...{.....
0000650: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0000660: 0000 0000 d483 1080 0000 0000 0000 0000 .....
0000670: 5c1f 1080 6721 1080 ff23 1080 8924 1080 \..g!...#...$.
0000680: 635e 1080 7555 1080 4d67 1080 5b65 1080 c^..uU..Mg...[e..
0000690: 0000 0000 0200 0000 0100 0000 0100 0000 .....
00006a0: acc8 1080 4c03 1180 0000 0000 e803 0000 ....L.....
00006b0: ad03 0000 c800 0000 1e00 0000 0200 0000 .....
00006c0: 2000 0000 3a00 0000 0000 0000 0000 0000 .....
00006d0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
```

Super Pages

- If the Page Size Extension (PSE) bit is set in CR4, we can optionally have entries in the page directory point directly to 4MB pages
- This can be beneficial because it reduces paging overhead (only one level of lookup, more data mapped)

Translation with Super Pages



32	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Address of page directory ¹																			Ignored					P C D	P W T	Ignored			CR3		
Bits 31:22 of address Pf 2MB page frame									Reserved (must be 0)			Bits 39:32 Of address			P A T	Ignored	G	1	0	A	P C D	P W T	U / S	R / W	1	PED: 4MB page					
Address of page table																			Ignored			0	I G N	A	P C D	P W T	U / S	R / W	1	PDE: page Table	
Ignored																											0	PDE: Not present			
Address of 4kb page frame															P A T	Ignored	G	P A T	D	A	P C D	P W T	U / S	R / W	1	PTE: 4KB page					
Ignored																											0	PTE: not present			

Running the Numbers

- Basics: super pages use 22 bits for offset because $2^{22} = 4\text{MB}$
- How many entries in a 4KB page table?
 - 1024 entries = 4MB addressed by one page table
 - So each page directory entry addresses 4MB of memory whether it points to a page table or a super page
- How many page directory entries?
 - We want to address 4GB of memory $\Rightarrow 4\text{GB}/4\text{MB} = 1024$
- In general that's why for x86 the available sizes are 4KB and 4MB pages

Example 3

- Q: Suppose one process has mapped 512MB worth of virtual address space using 4KB pages. How much space is required?
- Remember: Each page table covers 4MB of memory

Example 3

- We need the 1st level page table. 4KB
- We need to be able address 512 MB or memory. How many 2nd level page tables do we need since each addresses 4 MB? $512/4 = 128$
- What's the size of each page table? 4KB.
- How much total space for the Second level page tables? $128 * 4KB$
- Total:

= size of (1 page dir)	+	number of second level pages * size of (1 page table)	
= 4KB			128
	*	4KB	
- = 4KB + .5 MB
- = ~0.503 MB

Max Overhead

- What if we just used a one-level page table with 4KB pages?
 - $(4\text{GB} / 4\text{KB}) \text{ entries} * 4 \text{ bytes / entry} = 4\text{MB}$
- What if we used 4MB pages everywhere?
 - $(4\text{GB} / 4\text{MB}) \text{ entries} * 4 \text{ bytes} = 4096 \text{ bytes} = 4\text{KB}$